

Week 6: Intro to surveydown

September 30, 2026

WHY SURVEYDOWN?

Your project lives or dies on its survey. Every choice model we estimate later in the semester is only as good as the data the survey collects. The usual tools (Google Forms, Qualtrics) are point-and-click: nothing is reproducible, the data sits on someone else's server, and the features you need for conjoint — random designs, conditional logic, custom display — are limited or paywalled.

[surveydown](#) flips that. A **survey is code**: a Quarto document for the content, an R script for the app, and a PostgreSQL database you control for the responses. Because it's plain text, it can be version controlled, shared, and — importantly for us — written and edited by an agent.

	Google Forms	surveydown
Reproducible	no	yes
You control the data	no	yes
Free & open source	partly	yes
Conjoint-ready	no	yes

THE TWO FILES

Every survey is a folder with two files that work together:

File	What it defines
survey.qmd	Content: pages, text, images, questions
app.R	The Shiny app: database, server logic, launch

Content goes in `survey.qmd`; logic goes in `app.R`. Questions and pages are **defined** in the Quarto doc. Anything that **decides** what a respondent sees (showing, skipping, stopping, reacting to answers) goes inside the `server()` function in `app.R`.

A minimal survey.qmd

```
---
survey-settings:
  mode: preview
---

```{r}
library(surveydown)
```

--- welcome

# Welcome to our survey!

```{r}
sd_question(
 type = "mc",
 id = "penguins",
 label = "What's your favorite penguin?",
 option = c(
 "Adélie" = "adelie",
 "Chinstrap" = "chinstrap",
 "Gentoo" = "gentoo"
)
)
```

--- end

```{r}
sd_close()
```
```

- **Pages** start with three dashes and a page id (`--- welcome`). No closing tag; the next page marker ends the page.
- A **Next** button is added to every page automatically.
- `sd_close()` on the last page adds an “Exit Survey” button.

The standard app.R

```
library(surveydown)

db <- sd_db_connect() # connect to database
ui <- sd_ui()          # standard UI

server <- function(input, output, session) {
  # conditional logic / reactivity goes here
  sd_server(db)
}

shiny::shinyApp(ui = ui, server = server)
```

Run it with the **Run App** arrow at the top of `app.R`. `surveydown` doesn't always pick up saved edits while running — stop the app and run it again.

Starting a new survey

```
surveydown::sd_create_survey(
  template = "default",
  path = "path/to/my-survey"
)
```

Every survey gets its own folder. Browse the [templates](#), or build visually with the [studio app](#) via `sdstudio::launch()`.

SURVEY BASICS

YAML settings

All settings live in the YAML header of `survey.qmd`. No header means all defaults. Full list on the [Survey Settings](#) page.

| Key | What it does |
|---------------|---|
| mode | Where responses are saved (below) |
| show-previous | yes adds a Previous button |
| all-required | yes makes every question required |
| required | List of specific required question ids |
| theme | Any bootswatch theme name |
| barcolor | Progress bar color |
| barposition | top, bottom, or none |

`mode`, `show-previous`, and the `required` keys go under `survey-settings`; the `theme` and `bar` keys go under `theme-settings`. Custom styling: `theme: [united, custom.scss]`.

| mode | Responses saved to |
|----------|--------------------------------|
| preview | preview_data.csv in the folder |
| local | local_data.csv in the folder |
| database | Your database (the default) |

Build in preview, launch in database.

Questions

```
sd_question(
  type = "mc",
  id = "fruit",
  label = "1. Do you like fruit?",
  option = c(
    "Yes!" = "yes",
    "Kind of" = "kind_of",
    "No :(" = "no"
  )
)
```

Options are written "Label" = "value": respondents see the label, your data stores the value. The `id` becomes the column name in your data, so keep it short and unique.

| Type | What it is |
|-------------|-----------------------------------|
| mc | Multiple choice, pick one |
| mc_multiple | Multiple choice, pick many |
| mc_buttons | Pick one, large buttons |
| select | Drop-down menu |
| text | One line of open text |
| textarea | Block of open text |
| numeric | Numbers only |
| matrix | Many items, same options (Likert) |

More on the [question types](#) page.

Or define questions in `questions.yml` — the same fields as YAML, keyed by id — then call `sd_question("fruit")` in `survey.qmd`. See `demo-questions-yml`.

Navigation and images

- `sd_nav(label_next = "Continue")` changes one page's buttons.
- Images: plain html is the most predictable — `` inside a `<center>` tag.

CONDITIONAL LOGIC

Three functions, all placed in `server()` in `app.R`. Each takes one or more formulas of the form `condition ~ "target_id"`.

| Function | If the condition is TRUE... |
|---------------------------|--|
| <code>sd_show_if()</code> | Show a question or page |
| <code>sd_skip_if()</code> | Skip ahead to a page |
| <code>sd_stop_if()</code> | Stop navigation, with a message |

Use `sd_value("id")` to read a respondent's answer.

Show a follow-up question

```
sd_show_if(
  sd_value("penguins") == "other" ~ "penguins_other"
)
```

Both questions are defined in `survey.qmd`; the target stays hidden until the condition is met. `sd_is_answered("id")` is another handy condition.

Randomly show one of two pages

```
rand_val <- sample(c("A", "B"), 1)
sd_store_value(rand_val) # record which version
sd_show_if(
  rand_val == "A" ~ "page2a",
  rand_val == "B" ~ "page2b"
)
```

`sd_store_value()` saves the draw in your data, so you know which version each respondent saw. This is the seed of conjoint randomization.

Screen out a respondent

```
sd_skip_if(
  sd_value("vehicle_ownership") == "no" ~ "screenout"
)
```

...with a `--- screenout` page in `survey.qmd` saying they don't qualify.

STORING DATA

Responses go to any **PostgreSQL** database. We use [Supabase](#) (free, open source). Full details on the [Storing Data](#) page.

1. Create a Supabase account and a project (the project is your database; choose a strong password).
2. Click **Connect**, choose **Direct**, and find the **Transaction pooler** section for your credentials.
3. Run `surveydown::sd_db_config()` in the console and enter them. They're saved in a `.env` file in your survey folder.
4. Set mode: `database` in `survey.qmd` (or delete the mode line).
5. `sd_db_connect()` in `app.R` reads `.env` and connects.

Never share or commit `.env` — it holds your database password. View your responses locally with `sdstudio::Launch()`.

REACTIVITY

Reactivity means **changing something based on respondent input**.

For the general idea read the [Mastering Shiny chapter](#); for surveydown, the [reactivity page](#).

The easy way is `sd_output()` in `survey.qmd`, no server code needed:

| type = | Displays (for the penguin question) |
|------------------|-------------------------------------|
| "value" | The stored value: adélie |
| "label_option" | The chosen label: Adélie |
| "label_question" | The question text |

Drop it into text as inline R, e.g. `Hello, `r sd_output("name", type = "value")``.

The full way is Shiny: put `output$greeting <- renderText({...})` in `server()` and `textOutput("greeting")` in `survey.qmd`. `renderText()` re-runs every time the name answer changes.

HAVING AN AGENT BUILD A SURVEY

Because a survey is just two text files, an agent can write the whole thing. Be specific in your prompt — topic, question types, number of pages, conditional logic — and point it at [surveydown.org](#) so it uses the real functions. Then **verify it yourself**; "done" from an agent is a claim, not a fact:

- Run the app and click through **every** page.
- Trigger each branch of the conditional logic, not just the default path.
- Check each `id` is short, unique, and means something in your data.
- Open `preview_data.csv` and confirm every question landed in a column.
- Broken or wrong? Describe what you saw and have it fix that.

ERRORS YOU WILL HIT

| Symptom | Usual cause |
|---------------------------|---|
| Edits don't appear | App still running old version; stop & re-run |
| Can't connect to database | No <code>.env</code> ; run <code>sd_db_config()</code> |
| No data in database | Still in mode: <code>preview</code> |
| Conditional logic ignored | Condition written in <code>survey.qmd</code> , not <code>app.R</code> |
| Question never shows | Target id typo in the formula |
| Paths break | Opened a file, not the survey folder |

BEFORE NEXT CLASS

- Read the [blog post on conjoint in surveydown](#) — next week we generate conjoint questions and put them in a survey.
- Do [HW 5](#), due **Mon, Oct 5**.
- Start building your team's [Pilot Survey](#), due **Tue, Oct 13**.

You should be able to:

- Create a new survey and run it from its folder in Positron.
- Say which file holds content and which holds logic.
- Add pages and questions of several types, and change the theme.
- Show, skip, or randomize pages with conditional logic.
- Connect a Supabase database and switch to mode: `database`.
- Display a respondent's answer with `sd_output()`.